Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №3-4
«Аудит безопасности веб-приложения» и «Анализ уязвимостей веб-приложения с помощью OWASP ZAP»

по дисциплине

«Информационная безопасность»

Выполнили: Ястребов-Амирханов Алекси
Группа: P3432
Преподаватель: Рыбаков Степан Дмитриевич

Санкт-Петербург,

2026

Оглавление

1. Краткое резюме аудита безопасности 3

2. Архитектура приложения и моделирование угроз по методике STRIDE 3

2.1. Диаграмма потоков данных (DFD) 3

2.2. Анализ угроз по категориям STRIDE 4

3. Автоматизированное сканирование (DAST) с помощью OWASP ZAP 4

3.1. Методика и запуск сканирования 4

3.2. Результаты автоматизированного анализа 5

4. Ручная верификация и эксплуатация уязвимостей 5

4.1. SQL-инъекция (SQLi) в форме аутентификации 6

4.2. Межсайтовый скриптинг (DOM-based XSS) в поисковом запросе 6

4.3. Раскрытие спецификации закрытого API (Swagger UI) 7

4.4. Открытый каталог файлов и резервных копий (Directory Listing / FTP) 7

4.5. Неавторизованный дамп конфигурации и утечка версии сервера 8

5. Сводная таблица уязвимостей и оценка рисков (CVSS v3.1) 9

6. Рекомендации по устранению выявленных рисков 10

7. Лабораторная работа 4 10

8. Проведение автоматизированного сканирования (Quick Scan) 11

9. Детальный анализ выявленных уязвимостей 11

9.1. SQL-инъекция (SQL Injection) — Уровень High 11

9.2. Отсутствие токенов защиты от CSRF-атак — Уровень Medium 12

9.3. Отсутствие заголовка защиты от кликджекинга — Уровень Medium 13

9.4. Утечка информации о сервере и версии (Server Banner Leak) — Уровень Low 14

10. Сводная таблица найденных уязвимостей 14

11. Рекомендации по устранению выявленных рисков 15

12. Вывод 15

1. Краткое резюме аудита безопасности

В рамках настоящей лабораторной работы был проведен комплексный аудит защищенности веб-приложения OWASP Juice Shop (версия бэкенд-движка Express ^4.22.1, база данных SQLite под управлением ORM Sequelize). Аудит сочетал в себе методологию динамического анализа безопасности приложений (DAST — Dynamic Application Security Testing) и проактивное архитектурное моделирование угроз по методологии Microsoft STRIDE.

Цель аудита — практическое выявление архитектурных и программных уязвимостей, всесторонняя оценка их потенциального влияния на конфиденциальность, целостность и доступность сервиса в соответствии с классификатором OWASP Top 10:2021 и международными метриками CVSS v3.1, а также выработка обоснованных рекомендаций по устранению выявленных рисков.

2. Архитектура приложения и моделирование угроз по методике STRIDE

2.1. Диаграмма потоков данных (DFD)

Для всестороннего анализа защищенности была спроектирована и построена диаграмма потоков данных (Data Flow Diagram — DFD). На диаграмме выделены три ключевых структурных блока: клиентское веб-приложение (Single Page Application на базе фреймворка Angular), серверный бэкенд (Node.js / Express REST API) и хранилища постоянных данных (реляционная база данных SQLite и каталог файловой системы ./ftp/).

В архитектуре четко идентифицированы две границы доверия (Trust Boundaries): первая граница отделяет недоверенную внешнюю среду (глобальную сеть Интернет и браузер пользователя) от бэкенда; вторая граница разделяет серверный процесс обработки запросов и изолированные локальные хранилища данных.

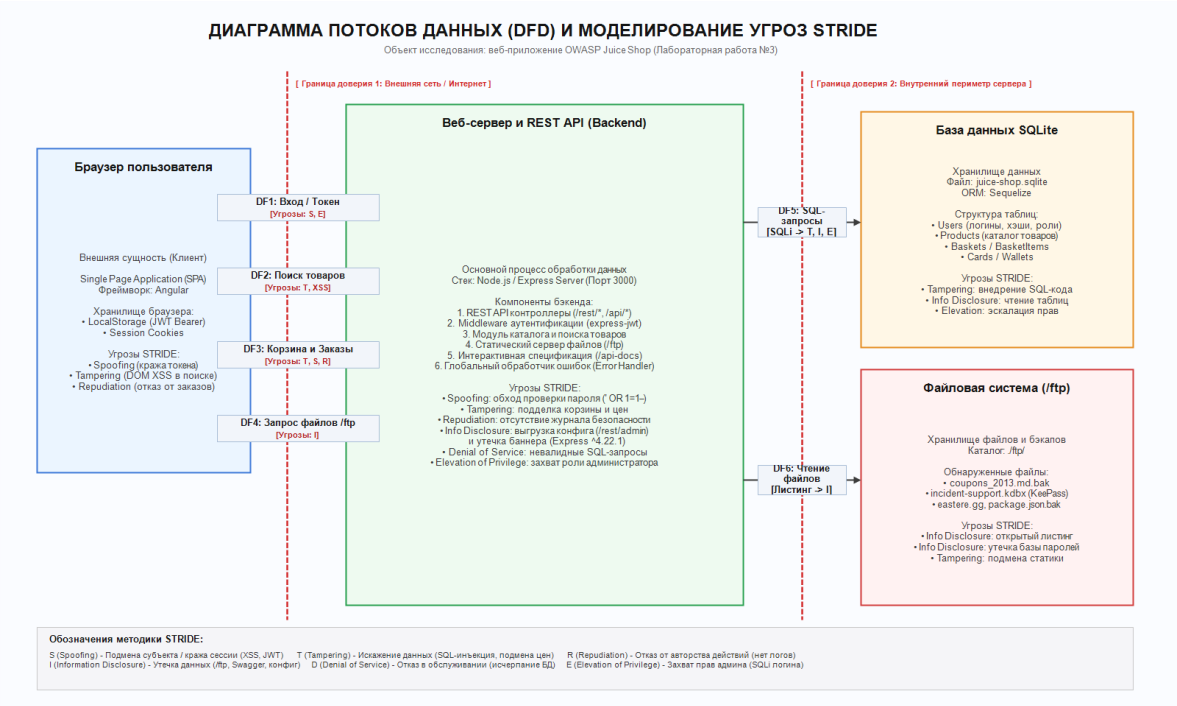


Рисунок 1 — Диаграмма потоков данных (DFD) веб-приложения OWASP Juice Shop с разметкой угроз по методике STRIDE

2.2. Анализ угроз по категориям STRIDE

Методика STRIDE позволила сопоставить компоненты системы и ключевые потоки данных с 6 классами угроз информационной безопасности:

• S (Spoofing — Подмена личности): На потоке DF1 (Аутентификация) угроза заключается в возможности несанкционированного входа под чужой учетной записью. На практике подтверждена критическая угроза полного обхода проверки подлинности через внедрение SQL-инъекции (' OR 1=1--), позволившая войти с правами суперпользователя admin без знания пароля. Кроме того, на потоке DF2 уязвимость XSS позволяет злоумышленнику осуществить кражу токена аутентификации (JWT) из localStorage браузера жертвы, что ведет к захвату сессии.

• T (Tampering — Искажение данных): На потоке DF5 (SQL-запросы к БД) угроза заключается во внедрении произвольного SQL-кода, способного модифицировать или удалить записи в БД. На потоке DF3 (Корзина и заказы) возможна манипуляция стоимостью товаров и содержимым корзин сторонних пользователей при отсутствии должной серверной валидации переданных идентификаторов.

• R (Repudiation — Отказ от совершенных действий): Отсутствие централизованного криптографически защищенного журнала аудита (audit logging) для критических бизнес-операций (оформление заказа, смена пароля, создание отзывов) позволяет недобросовестному пользователю отрицать факт совершения деструктивных или финансовых операций.

• I (Information Disclosure — Раскрытие информации): В приложении выявлены множественные векторы утечки чувствительных сведений: на потоке DF4 через открытый листинг каталога /ftp доступны бэкапы и файл базы паролей incident-support.kdbx; через открытый эндпоинт /rest/admin/application-configuration отдается вся конфигурация системы; через ответы об ошибках (401/500) утекают точные версии библиотек (Express ^4.22.1, SQLite, Sequelize).

• D (Denial of Service — Отказ в обслуживании): Отсутствие ограничений интенсивности запросов (rate limiting) и возможность передачи синтаксически некорректных тяжелых конструкций в поисковые запросы способны приводить к исчерпанию ресурсов сервера и блокировке цикла событий Node.js (ReDoS / DoS).

• E (Elevation of Privilege — Повышение привилегий): На потоке DF1 неавторизованный пользователь без зарегистрированной учетной записи успешно эскалирует свои полномочия до роли системного администратора (admin@juice-sh.op) за счет внедрения SQL-конструкции в форму авторизации.

3. Автоматизированное сканирование (DAST) с помощью OWASP ZAP

3.1. Методика и запуск сканирования

Для проведения динамического аудита безопасности был развернут локальный контейнер с веб-приложением OWASP Juice Shop на порту 3000. В качестве DAST-сканера использовался инструментарий OWASP ZAP (Zed Attack Proxy) версии 2.17.0. Сканирование включало в себя предварительный краулинг структуры сайта с помощью традиционного поискового паука (Spider) и специализированного AJAX-паука (для динамического рендеринга SPA на Angular), а также активное сканирование (Active Scan) с политикой Dev Standard.

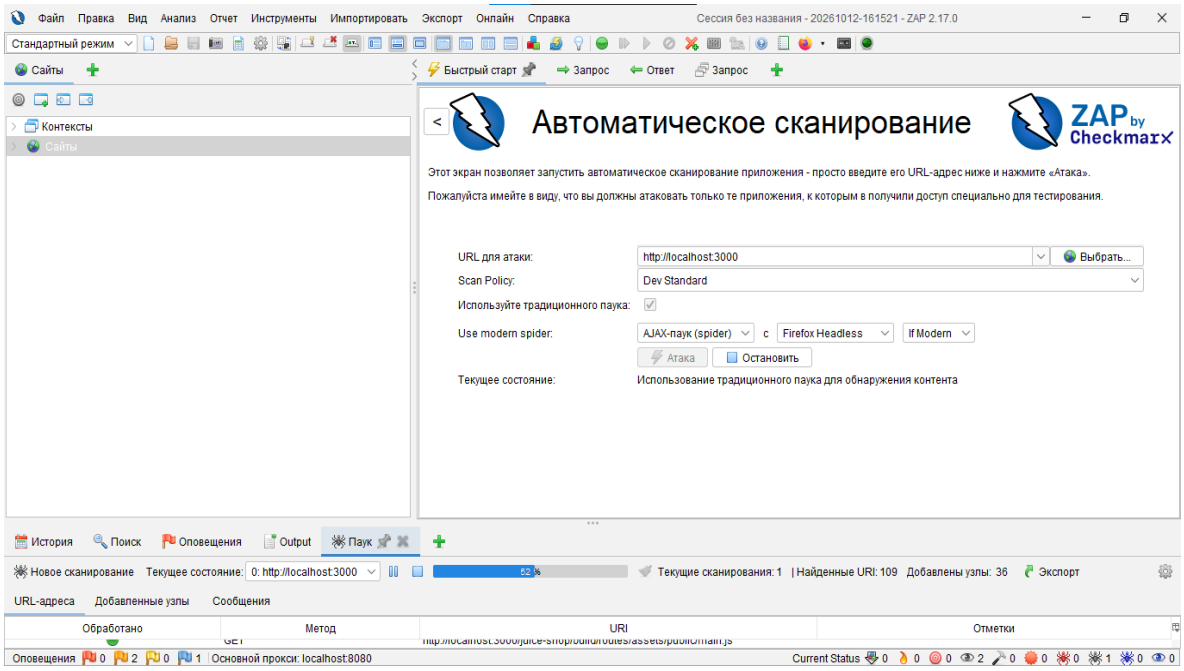


Рисунок 2 — Процесс автоматического сканирования стенда в OWASP ZAP (AJAX-паук и Active Scan)

3.2. Результаты автоматизированного анализа

По завершении активного сканирования сканер зафиксировал более 2100 тестовых запросов и сформировал дерево предупреждений (вкладка «Оповещения»). Были обнаружены уязвимости категорий High (SQL-инъекция), Medium (отсутствие заголовка Content Security Policy, проблемы сессионных ID, кликджекинг) и Low (отсутствие заголовков защиты от MIME-sniffing X-Content-Type-Options, утечка меток времени Unix).

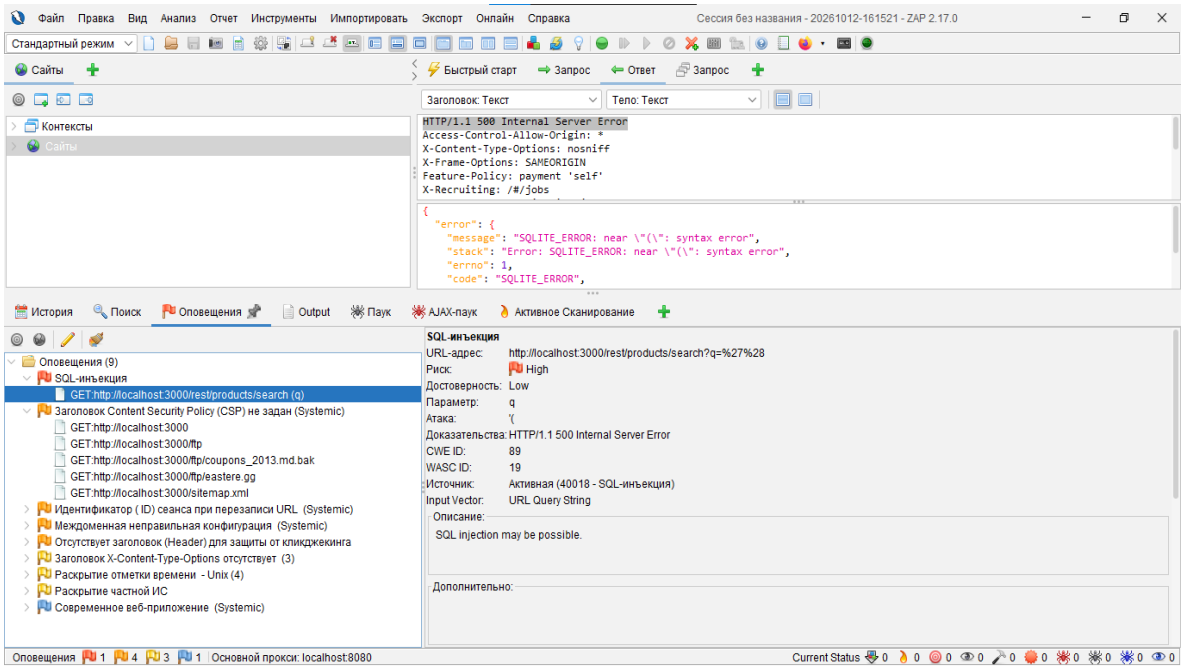


Рисунок 3 — Сводная вкладка «Оповещения» в OWASP ZAP с обнаруженными уязвимостями (включая SQLi со стектрейсом ошибки SQLite)

4. Ручная верификация и эксплуатация уязвимостей

По результатам работы сканера и анализа архитектуры было проведено ручное тестирование на проникновение для подтверждения 5 ключевых уязвимостей.

4.1. SQL-инъекция (SQLi) в форме аутентификации

Описание: Поле ввода адреса электронной почты на странице авторизации (/#/login) конкатенируется напрямую в SQL-запрос проверки учетных данных без параметризации (Prepared Statements).

Шаги воспроизведения (PoC): В поле «Email» вводится полезная нагрузка: ' OR 1=1--, в поле «Password» вводится произвольный символ. Символы -- комментируют остаток SQL-запроса проверки пароля. В результате условие 1=1 становится истинным, и СУБД возвращает первую попавшуюся запись таблицы Users, которой является суперпользователь admin@juice-sh.op.

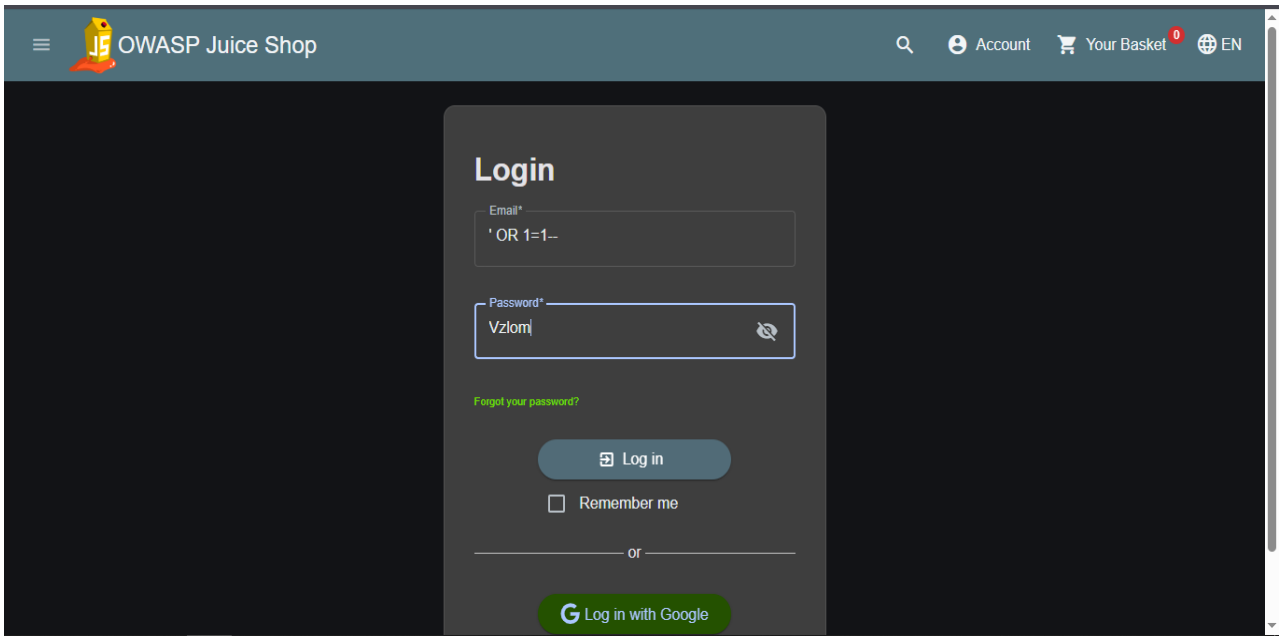


Рисунок 4 — Внедрение SQL-пейлоада в форму аутентификации

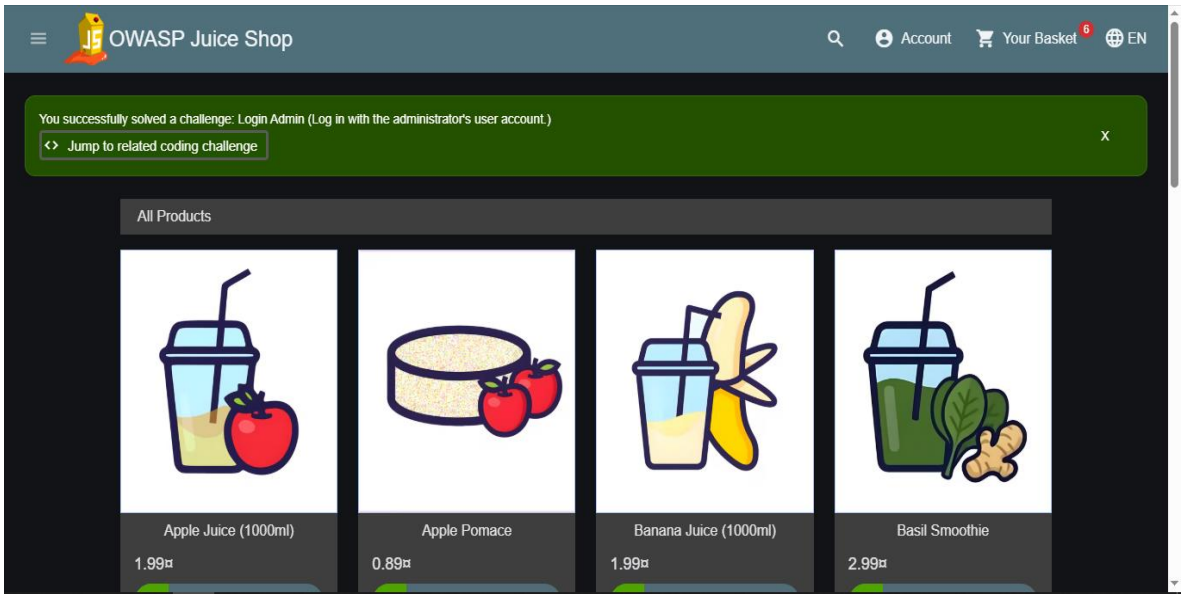


Рисунок 5 — Успешный обход аутентификации: получение сессии администратора и решение челленджа «Login Admin»

4.2. Межсайтовый скриптинг (DOM-based XSS) в поисковом запросе

Шаги воспроизведения (PoC): В строку глобального поиска вводится пейлоад: <iframe src="javascript:alert('XSS')">. При выполнении поиска браузер исполняет внедренный фрейм и вызывает системную функцию alert с сообщением «XSS».

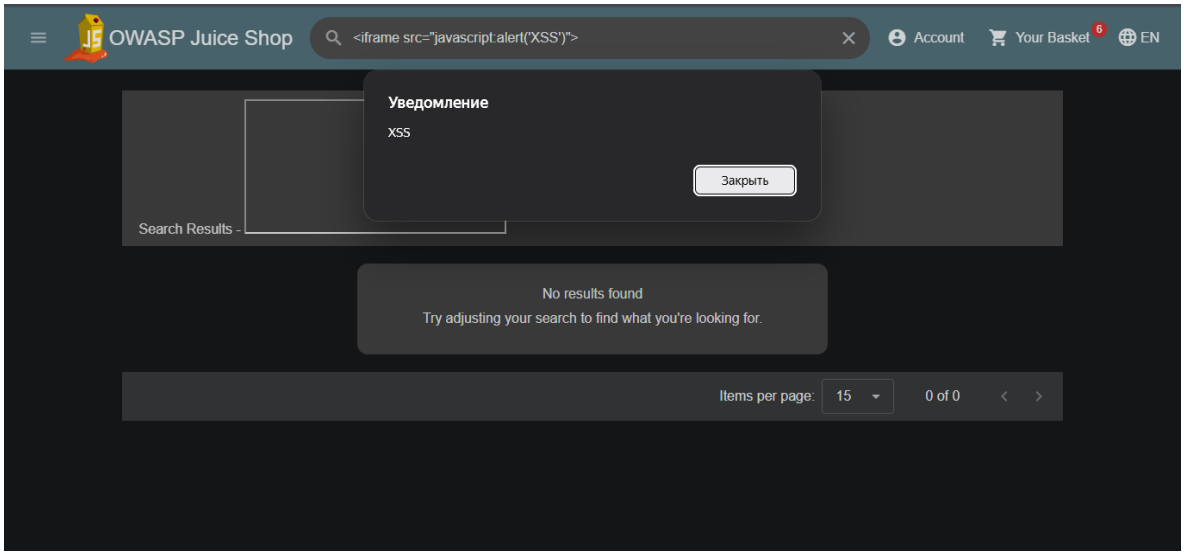


Рисунок 6 — Срабатывание модального окна JavaScript alert при эксплуатации DOM-based XSS

4.3. Раскрытие спецификации закрытого API (Swagger UI)

Шаги воспроизведения (PoC): При прямом переходе по URL http://localhost:3000/api-docs/ открывается Swagger UI с детальным описанием методов B2B API, схем запросов и структуры сущностей (Order, Basket и др.).

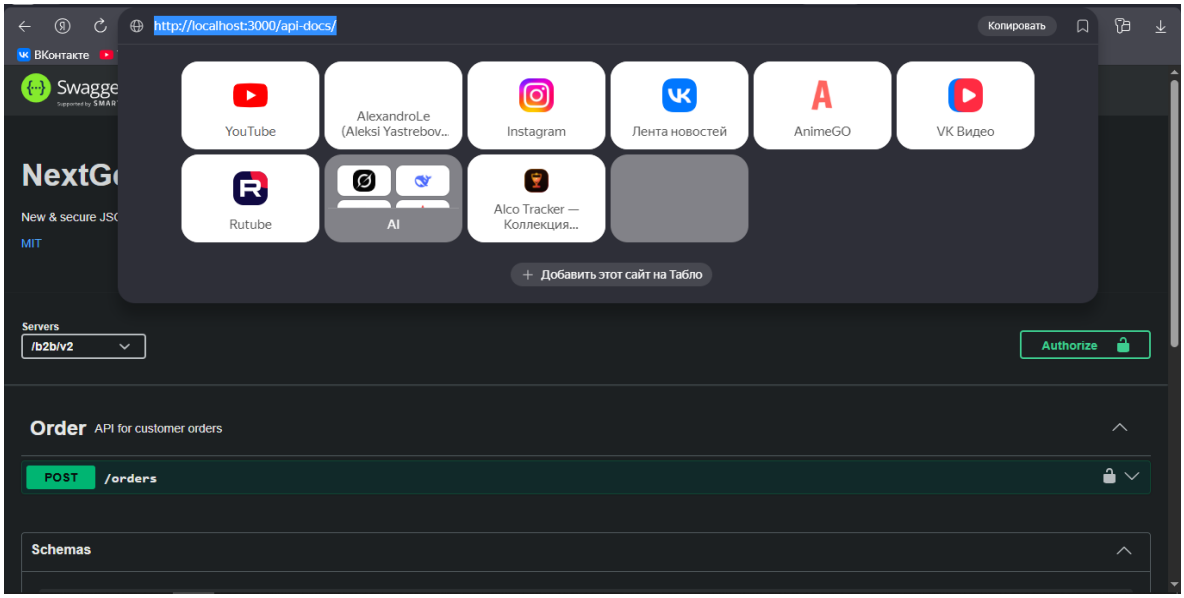


Рисунок 7 — Публично доступная документация внутренних REST API эндпоинтов (Swagger UI)

4.4. Открытый каталог файлов и резервных копий (Directory Listing / FTP)

Шаги воспроизведения (PoC): При переходе по адресу http://localhost:3000/ftp браузер отображает список файлов: coupons_2013.md.bak, eastere.gg, package.json.bak, а также критический файл incident-support.kdbx, содержащий пароли технической поддержки, доступные для немедленной загрузки.

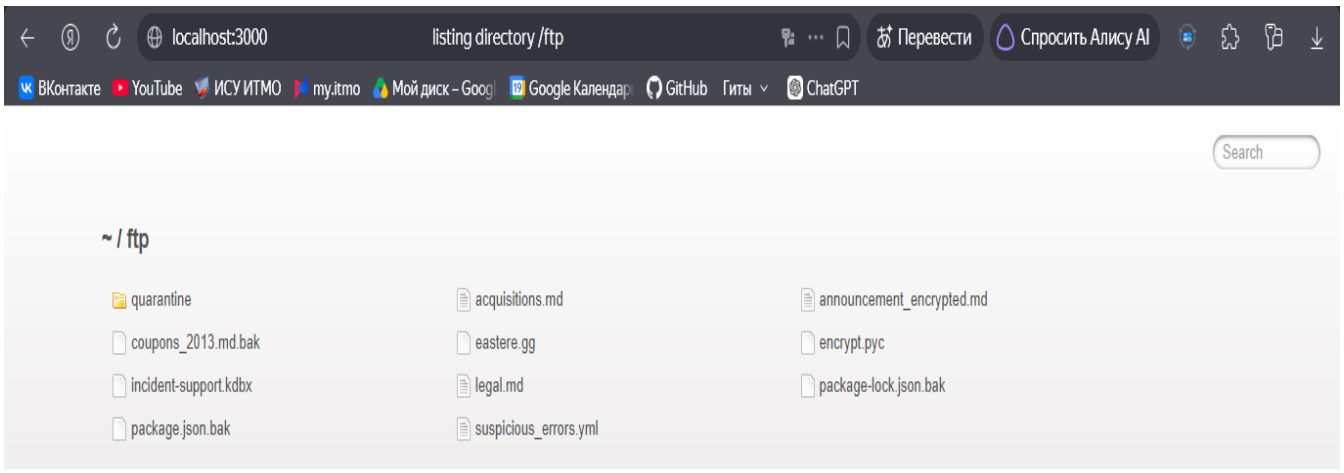


Рисунок 8 — Открытый листинг каталога /ftp с резервными копиями и базой паролей incident-support.kdbx

4.5. Неавторизованный дамп конфигурации и утечка версии сервера

Шаги воспроизведения (PoC): Обращение к маршруту http://localhost:3000/rest/admin/application-configuration без авторизационных заголовков возвращает полный JSON-дамп внутренних параметров безопасности, секретов и ответов на контрольные вопросы. При обращении к закрытым маршрутам без токена сервер открыто декларирует баннер Express ^4.22.1 и имя внутреннего класса UnauthorizedError.

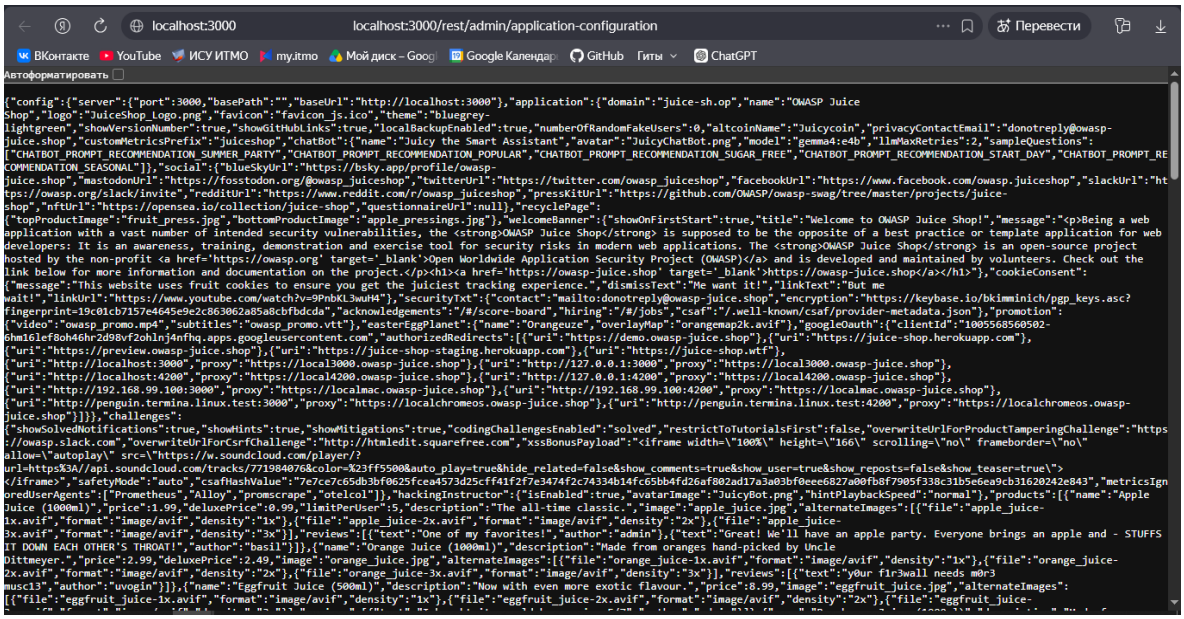


Рисунок 9 — Неавторизованная выгрузка закрытой конфигурации приложения (/rest/admin/application-configuration)

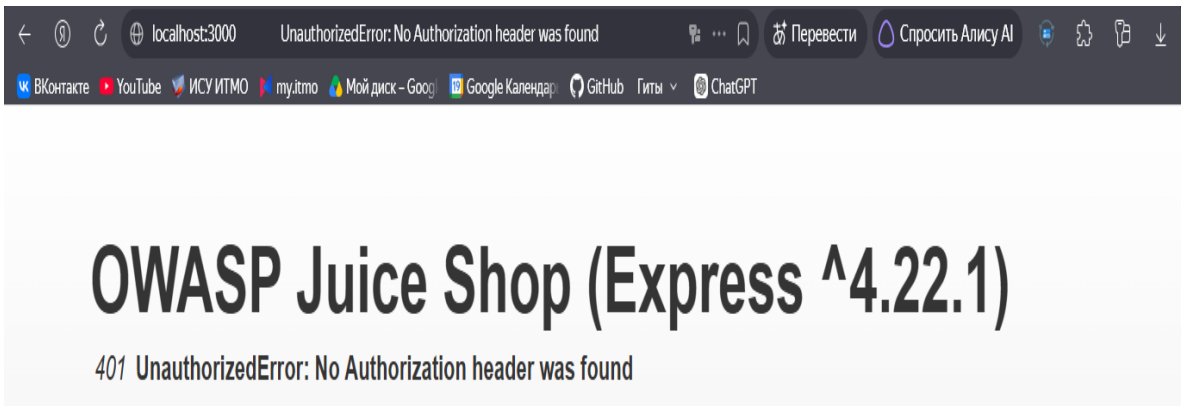


Рисунок 10 — Раскрытие версий ПО (Express ^4.22.1) и системного типа ошибки в ответе сервера

5. Сводная таблица уязвимостей и оценка рисков (CVSS v3.1)

Для всех верифицированных уязвимостей произведена классификация по стандарту OWASP Top 10:2021 и выполнен расчет базового балла по международному калькулятору CVSS v3.1 (Common Vulnerability Scoring System) от FIRST.org.

№ / Уязвимость	Категория OWASP Top 10	Оценка CVSS v3.1	Краткое описание и PoC	Меры устранения
1. SQL Injection (обход авторизации)	A03:2021 – Injection	9.8 Critical CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	Внедрение ' OR 1=1-- в форму входа позволяет войти под учетной записью суперпользователя без знания пароля.	Использование параметризованных запросов (Prepared Statements) или безопасных методов ORM Sequelize.
2. DOM-based Cross-Site Scripting (XSS)	A03:2021 – Injection	6.1 Medium CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	Ввод тега <iframe src="javascript:alert('XSS')"> в поисковую строку приводит к выполнению произвольного JS-кода в сессии.	Санитизация и экранирование входных данных, внедрение строгой политики Content Security Policy (CSP).
3. Публичный листинг файлов (/ftp)	A01:2021 – Broken Access Control	7.5 High CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	Каталог /ftp открыт для чтения, содержит бэкапы и файл базы паролей KeePass (incident-support.kdbx).	Отключение автоматического листинга директорий (directory listing), ограничение доступа к служебным файлам.
4. Раскрытие внутренней конфигурации	A01:2021 – Broken Access Control	7.5 High CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	Эндпоинт /rest/admin/application-configuration отдает всю структуру приложения и секреты без авторизации.	Внедрение строгой ролевой модели (RBAC) и проверка валидности JWT-токена администратора на бэкенде.

№ / Уязвимость	Категория OWASP Top 10	Оценка CVSS v3.1	Краткое описание и PoC	Меры устранения
5. Утечка версий ПО и ошибок (Information Disclosure)	A05:2021 – Security Misconfiguration	5.3 Medium CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	В заголовках страниц и текстах ошибок 401/500 сервер раскрывает стек технологий (Express ^4.22.1, классы ошибок).	Отключение баннеров сервера (app.disable('x-powered-by')), настройка универсальных кастомных страниц ошибок.

6. Рекомендации по устранению выявленных рисков

На основе проведенного DAST-анализа и моделирования угроз STRIDE разработаны следующие ключевые рекомендации по усилению защищенности:

1. Внедрение параметризованных запросов: категорически исключить конкатенацию пользовательских строк в SQL-запросы. Использовать безопасные методы ORM (Sequelize) с автоэкранированием входных параметров.

2. Внедрение строгой политики Content Security Policy (CSP): добавить HTTP-заголовок Content-Security-Policy (например, default-src 'self'; script-src 'self'; object-src 'none'), блокирующий исполнение inline-скриптов и несанкционированную загрузку сторонних ресурсов.

3. Усиление контроля доступа (RBAC): обеспечить обязательную валидацию JWT-токенов в промежуточном слое (middleware) перед вызовом любых административных API (/rest/admin/*) и приватных маршрутов корзины.

4. Запрет Directory Listing и изоляция служебных файлов: отключить модуль листинга директорий в веб-сервере. Вынести резервные копии (.bak), служебные дампы и базы паролей (.kdbx) за пределы корневой папки веб-приложения.

5. Безопасная обработка ошибок и сокрытие версий (Hardening): отключить заголовок X-Powered-By, скрыть версию Express в глобальных обработчиках ошибок, настроить возврат стандартизированных JSON-ответов об ошибках без вывода трассировочного стека (stack trace).

7. Лабораторная работа 4

Цель лабораторной работы — освоить базовые практические навыки динамического тестирования защищенности веб-приложений (DAST — Dynamic Application Security Testing) с использованием специализированного сканера OWASP ZAP (Zed Attack Proxy) версии 2.17.0.

В качестве объекта исследования был выбран публичный тестовый веб-полигон Altoro Mutual (http://demo.testfire.net/), созданный для безопасного проведения аудитов безопасности и моделирования атак. Приложение представляет собой онлайн-банкинг, функционирующий на сервере приложений Apache Tomcat / Apache-Coyote с JSP/Java-бэкендом.

8. Проведение автоматизированного сканирования (Quick Scan)

Сканирование было инициировано через модуль автоматического аудита ZAP. В поле «URL для атаки» был указан адрес http://demo.testfire.net/. Сканер последовательно провел рекурсивный обход ссылок (паук / spider) и выполнил активное тестирование эндпоинтов на типовые уязвимости веб-приложений.

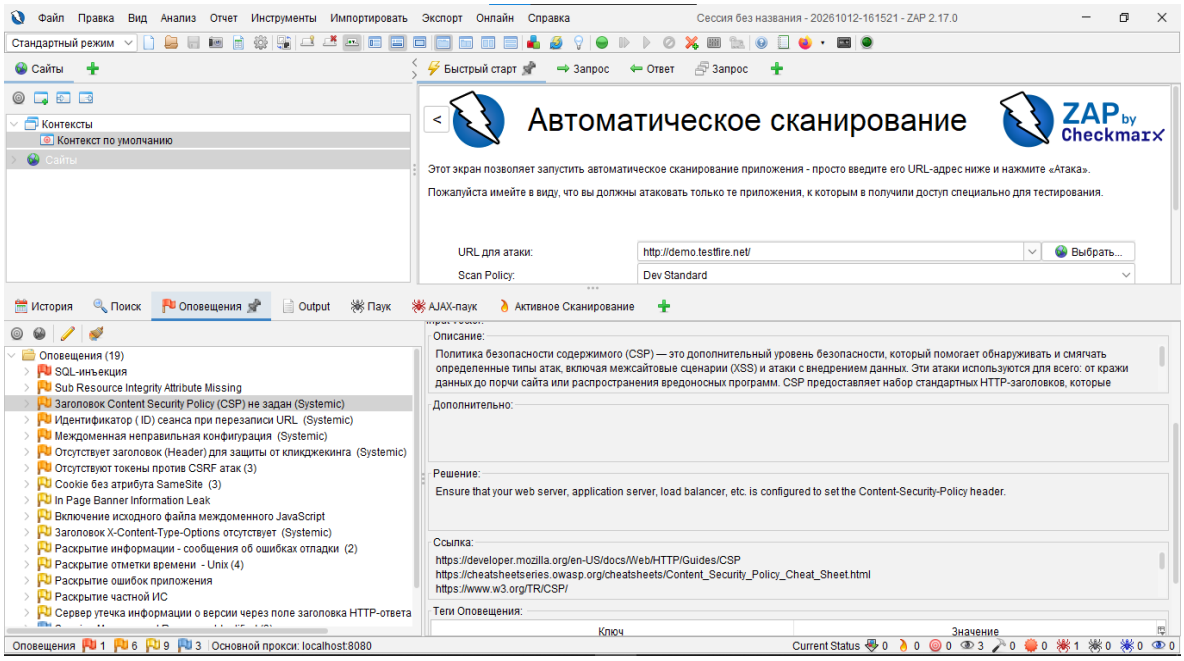


Рисунок 1 — Сводная панель оповещений OWASP ZAP по результатам сканирования demo.testfire.net

9. Детальный анализ выявленных уязвимостей

Для детального анализа были выбраны 4 ключевые уязвимости различных категорий риска:

9.1. SQL-инъекция (SQL Injection) — Уровень High

Идентификатор CWE: CWE-89 (Improper Neutralization of Special Elements used in an SQL Command).

Описание: Сканер ZAP обнаружил, что параметры поисковых и пользовательских запросов конкатенируются напрямую в SQL-выражения без должной параметризации. При отправке специального символа кавычки ' сервер вернул ошибку выполнения SQL-запроса (HTTP 500 Internal Server Error).

Опасность: Злоумышленник может модифицировать логику выполнения запросов, полностью обойти процесс аутентификации, выгрузить конфиденциальные данные пользователей банка или изменить баланс счетов.

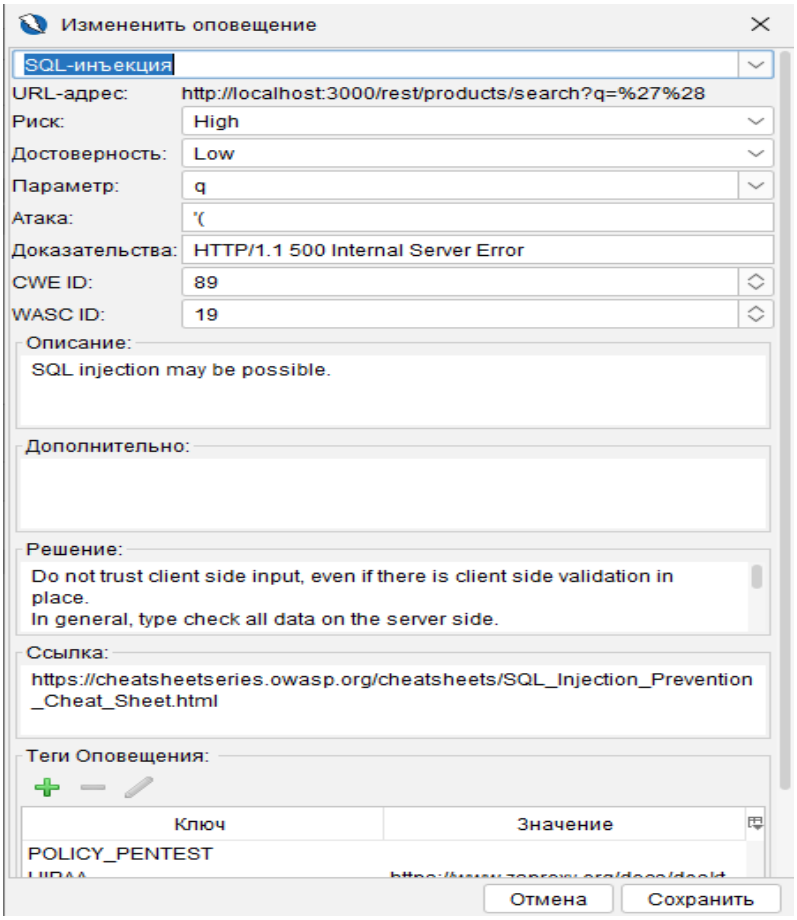


Рисунок 2 — Карточка уязвимости SQL-инъекция в OWASP ZAP (CWE-89, High Risk)

9.2. Отсутствие токенов защиты от CSRF-атак — Уровень Medium

Идентификатор CWE: CWE-352 (Cross-Site Request Forgery).

Описание: В форме отправки обратной связи (/feedback.jsp) отсутствуют уникальные непредсказуемые токены защиты от подделки межсайтовых запросов (anti-CSRF tokens). Запрос формируется с использованием стандартного метода POST без скрытых полей проверки подлинности источника.

Опасность: Злоумышленник может создать вредоносную веб-страницу, при посещении которой браузер авторизованной жертвы незаметно отправит HTTP-запрос на выполнение непреднамеренного действия в банковском приложении.

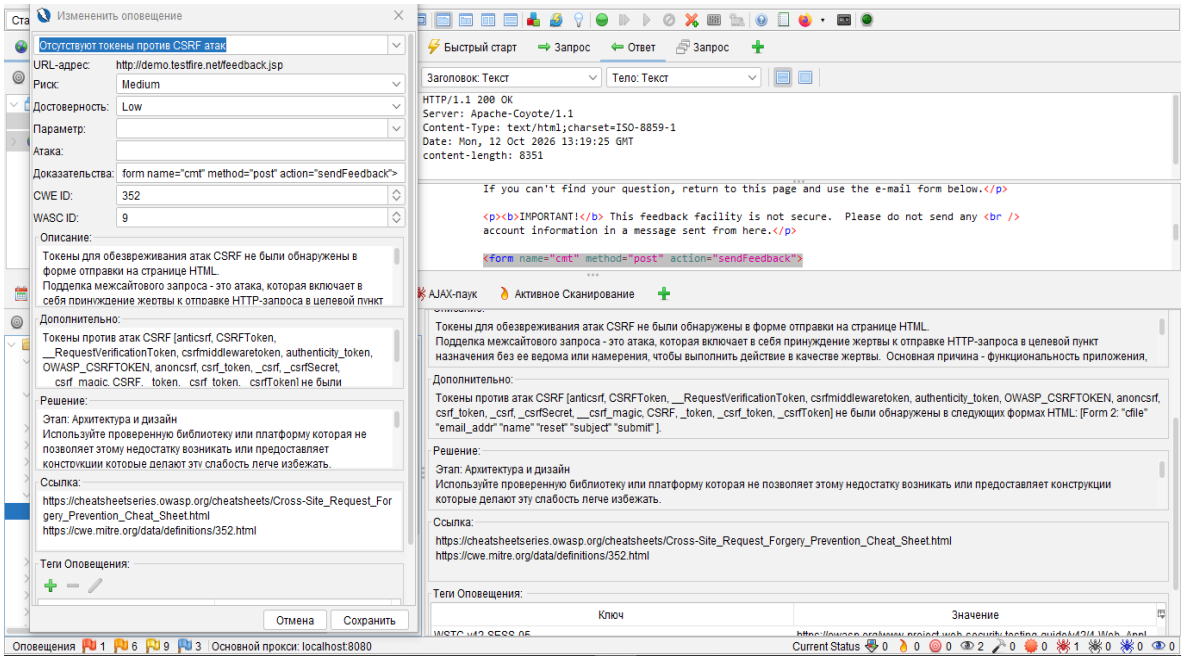


Рисунок 3 — Карточка уязвимости отсутствия CSRF-токенов в форме feedback.jsp (CWE-352, Medium Risk)

9.3. Отсутствие заголовка защиты от кликджекинга — Уровень Medium

Идентификатор CWE: CWE-1021 (Improper Restriction of Rendered UI Layers or Frames).

Описание: В HTTP-ответах сервера на корневой URL http://demo.testfire.net/ отсутствует заголовок X-Frame-Options, а также директива frame-ancestors в заголовке Content-Security-Policy.

Опасность: Приложение может быть встроено в прозрачный фрейм (iframe) на стороннем вредоносном ресурсе. Пользователь, полагая, что взаимодействует с доверенной кнопкой, невольно кликает по скрытым элементам интерфейса банка (Clickjacking), например, подтверждая то что продает душу компании злоумышленника.

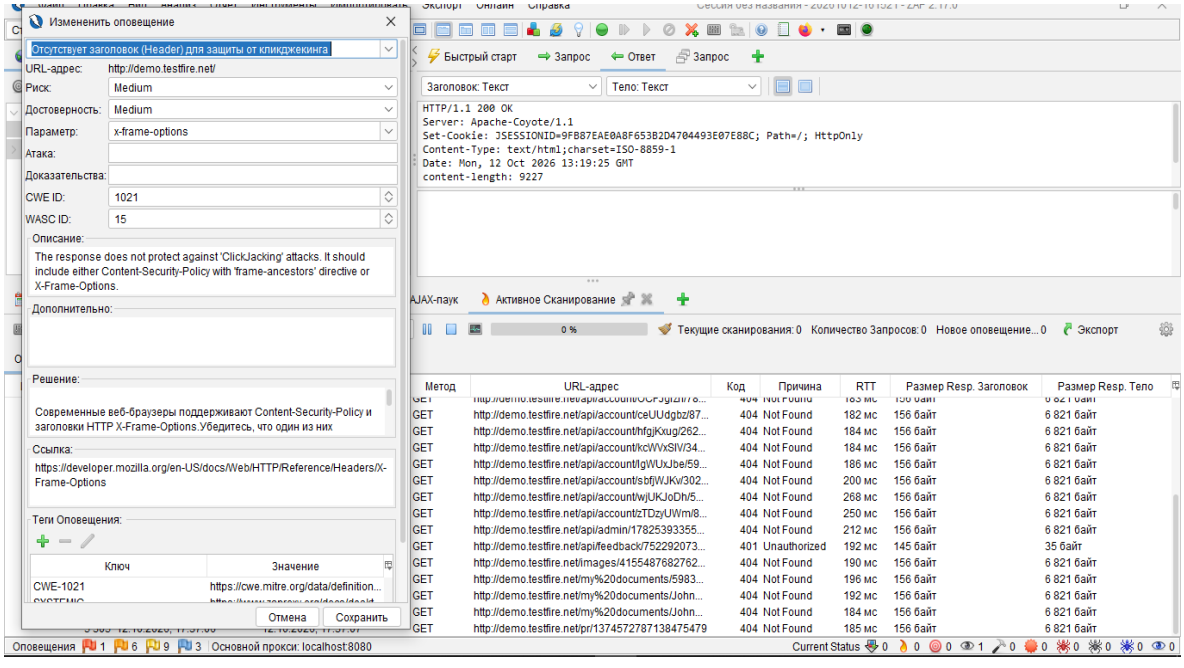


Рисунок 4 — Карточка уязвимости отсутствия защиты от кликджекинга (CWE-1021, Medium Risk)

9.4. Утечка информации о сервере и версии (Server Banner Leak) — Уровень Low

Идентификатор CWE: CWE-497 (Exposure of Sensitive System Information to an Unauthorized Control Sphere).

Описание: Сервер передает точную идентификационную информацию о версии своего программного обеспечения в стандартном HTTP-заголовке ответа: Server: Apache-Coyote/1.1.

Опасность: Предоставление точной версии ПО существенно облегчает злоумышленнику этап разведки (Reconnaissance), позволяя осуществлять целевой подбор эксплойтов под конкретные известные уязвимости (CVE) сервера Apache Tomcat.

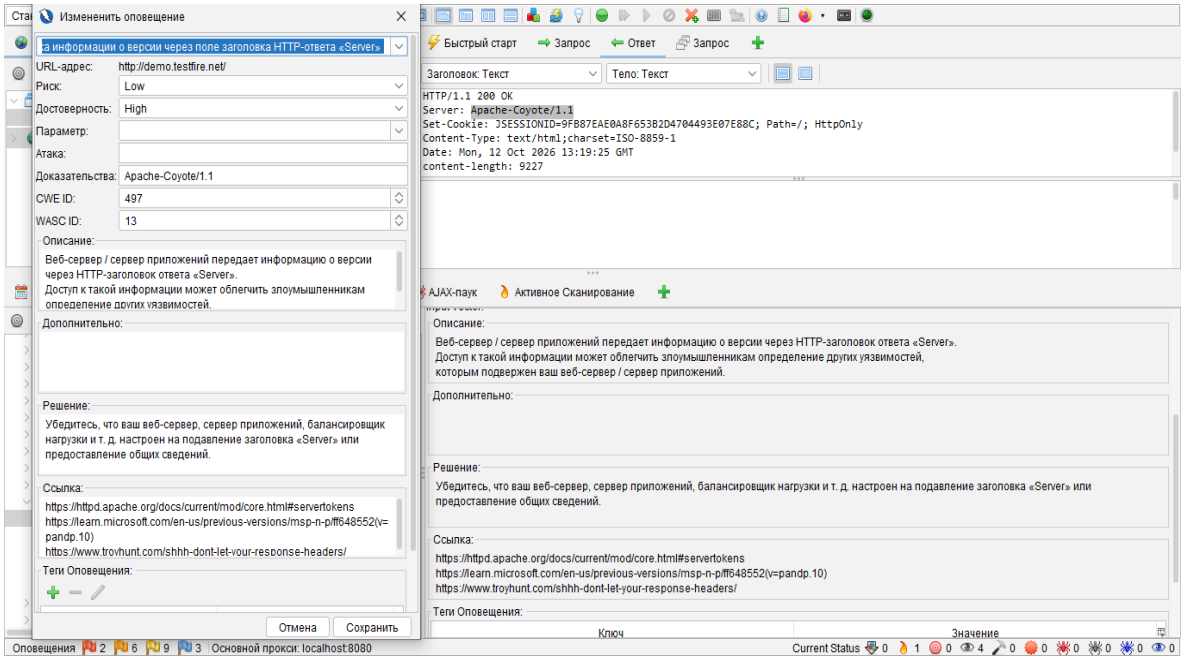


Рисунок 5 — Карточка уязвимости раскрытия версии сервера в HTTP-заголовке (CWE-497, Low Risk)

10. Сводная таблица найденных уязвимостей

Ниже приведена итоговая сводка классификации проанализированных уязвимостей в соответствии с OWASP Top 10 и CWE:

Уязвимость	CWE ID / Уровень	Категория OWASP Top 10	Суть угрозы	Рекомендации по устранению
SQL Injection	CWE-89 High	A03:2021 – Injection	Конкатенация параметров ввода в SQL-запрос; обход авторизации, кража данных БД.	Использование параметризованных запросов (Prepared Statements), ORM, экранирование спецсимволов.
Отсутствие CSRF-токенов	CWE-352 Medium	A01:2021 – Broken Access Control	Выполнение несанкционированных действий от имени авторизованного пользователя через сторонние страницы.	Внедрение уникальных анти-CSRF токенов (Synchronizer Token Pattern) и атрибута SameSite=Lax/Strict для Cookies.

Уязвимость	CWE ID / Уровень	Категория OWASP Top 10	Суть угрозы	Рекомендации по устранению
Отсутствие защиты от кликджекинга	CWE-1021 Medium	A05:2021 – Security Misconfiguration	Отображение страниц банка в невидимых iframe на фишинговых сайтах для перехвата кликов.	Добавление заголовка X-Frame-Options: DENY (или SAMEORIGIN) и директивы frame-ancestors 'none' в CSP.
Утечка версии сервера (Server banner)	CWE-497 Low	A05:2021 – Security Misconfiguration	Раскрытие баннера Server: Apache-Coyote/1.1, упрощающее таргетированный поиск эксплойтов.	Подавление заголовка Server или подмена его на нейтральное значение в конфигурации Apache Tomcat.

11. Рекомендации по устранению выявленных рисков

Для повышения уровня защищенности веб-ресурса рекомендуется реализовать комплекс превентивных мер:

1. Параметризация SQL: Перевести все запросы к базе данных на использование подготовленных выражений (Prepared Statements). Никогда не использовать строковую конкатенацию для формирования тела запроса.

2. Защита от CSRF: Для всех форм и операций, изменяющих состояние системы (POST, PUT, DELETE), внедрить проверку динамических CSRF-токенов. Установить для сессионных куки директиву SameSite=Lax.

3. Защитные HTTP-заголовки: Внедрить строгую политику Content-Security-Policy, запрещающую встраивание во фреймы (frame-ancestors 'self'), и установить заголовок X-Frame-Options: SAMEORIGIN.

4. Харденинг веб-сервера: Отключить отправку заголовков с версиями ПО (Server, X-Powered-By) в конфигурационных файлах сервера Tomcat (server.xml).

12. Вывод

В ходе выполнения лабораторной работы 3 были успешно освоены методы динамического тестирования защищенности (DAST) с использованием сканера OWASP ZAP и методология моделирования угроз STRIDE. Проведена практическая эксплуатация 5 критических уязвимостей, рассчитаны оценки рисков по шкале CVSS v3.1 и сформированы рекомендации по повышению защищенности сервиса.

В ходе лабораторной работы 4 были успешно освоены базовые практические навыки динамического тестирования защищенности веб-ресурсов с использованием сканера OWASP ZAP на примере тестового полигона demo.testfire.net. Были зафиксированы и проанализированы уязвимости категорий SQLi, CSRF, Clickjacking и Information Leakage, выполнена их классификация по OWASP Top 10 и подготовлены рекомендации по защите сервиса.